

claude-wsl-ubuntu / 4f364eb7-a600-4a2c-9b37-e5a313b6db9d

Metadata

```
- Source: `claude-wsl-ubuntu`
- Kind: `claude`
- Source file: `\\wsl.localhost\Ubuntu\home\avidullu\.claude\projects\home-avidullu-projects-khelsutra-guru\4f364eb7-a600-4a2c-9b37-e5a313b6db9d\subagents\workflows\wf_b0aa0abe-638\agent-a202351c55a0ceb23.jsonl`
- SHA-256: `53b353a8f6175a2ea8b035d79cb23bef8a351b24e3b6499542daf00f5dbb0442`
- Source modified: `2026-06-29T17:05:47+00:00`
- Imported at: `2026-07-05T16:48:30+00:00`
- project: `wf_b0aa0abe-638`
- session_id: `4f364eb7-a600-4a2c-9b37-e5a313b6db9d`
```

Transcript

1. user (2026-06-29T17:01:41.451Z)

In /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer, produce the fix plan for issue #397: stray scheme-prefixed dirs (`gcs:/`, `gs:/`, `local:/`) get written to the REPO ROOT when a gcs://, gs://, or local:// URI is resolved to a LITERAL local path instead of via the storage layer.

Find the WRITER(s): grep backend/storage/ref.py (resolve/fetch/resolve_input_uri), the per-video workspace / report-writer code, and any os.makedirs/open on a uri-like string. The `local:/tmp/pytest-of-avidullu/...` tree comes from a test like test_process_local_scheme_uri ? find it.

Determine: (a) the exact writer file:line + WHY it yields a literal 'gcs:/...' path (e.g. os.path.join on a 'gcs://bucket/..' string, or a local:// scheme stripped to 'local:/...'); (b) the minimal, low-risk code fix (route through the storage layer, or normalize the scheme to a proper local dir under output/); (c) the exact .gitignore lines to block recurrence (the colon-scheme dirs); (d) confirm the existing dirs are regenerable junk safe to delete. Identify a unit test to add that asserts no scheme-prefixed dir is created at repo root. Keep the code fix surgical.

2. assistant (2026-06-29T17:01:44.723Z)

I'll investigate issue #397 about stray scheme-prefixed directories being written to the repo root. Let me start by exploring the codebase structure.

3. user (2026-06-29T17:01:47.786Z)

```
total 200
drwxr-xr-x 22 avidullu avidullu 4096 Jun 27 17:27 .
drwxr-xr-x 12 avidullu avidullu 4096 Jun 29 13:02 ..
-rw-r--r-- 1 avidullu avidullu 598 Jun 25 11:46 .coveragerc
drwxr-xr-x 8 avidullu avidullu 4096 Jun 29 16:49 .git
-rw-r--r-- 1 avidullu avidullu 932 Jun 25 11:46 .gitattributes
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 11:46 .github
-rw-r--r-- 1 avidullu avidullu 1331 Jun 25 16:06 .gitignore
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 13:12 .mypy_cache
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 13:10 .pytest_cache
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 13:12 .ruff_cache
drwxr-xr-x 2 avidullu avidullu 4096 Jun 25 11:46 .vscode
-rw-r--r-- 1 avidullu avidullu 5498 Jun 27 17:27 CLAUDE.md
-rw-r--r-- 1 avidullu avidullu 34880 Jun 25 11:46 README.md
drwxr-xr-x 2 avidullu avidullu 4096 Jun 29 10:51 __pycache__
drwxr-xr-x 20 avidullu avidullu 4096 Jun 27 17:27 backend
-rw-r--r-- 1 avidullu avidullu 2650 Jun 25 11:46 config.json
drwxr-xr-x 5 avidullu avidullu 4096 Jun 25 11:46 deploy
drwxr-xr-x 6 avidullu avidullu 4096 Jun 29 14:05 docs
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 11:46 eval_baselines
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 13:11 gcs:
```

```

drwxr-xr-x 4 avidullu avidullu 4096 Jun 25 13:11 gs:
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 13:11 local:
-rw-r--r-- 1 avidullu avidullu 1960 Jun 25 11:46 mypy.ini
drwxr-xr-x 4 avidullu avidullu 4096 Jun 29 11:12 output
-rw-r--r-- 1 avidullu avidullu 4748 Jun 27 17:27 pyproject.toml
-rw-r--r-- 1 avidullu avidullu 240 Jun 25 11:46 pytest.ini
-rw-r--r-- 1 avidullu avidullu 2003 Jun 27 17:27 requirements.txt
-rw-r--r-- 1 avidullu avidullu 789 Jun 25 11:46 ruff.toml
-rw-r--r-- 1 avidullu avidullu 2051 Jun 27 17:27 run_all_tests.py
-rw-r--r-- 1 avidullu avidullu 5572 Jun 27 17:27 run_eval.py
-rw-r--r-- 1 avidullu avidullu 4457 Jun 27 17:27 run_regression.py
drwxr-xr-x 2 avidullu avidullu 4096 Jun 25 11:46 scratch
drwxr-xr-x 3 avidullu avidullu 4096 Jun 27 17:27 scripts
drwxr-xr-x 3 avidullu avidullu 12288 Jun 29 14:05 tests
drwxr-xr-x 3 avidullu avidullu 4096 Jun 27 17:27 tools
drwxr-xr-x 4 avidullu avidullu 4096 Jun 25 13:10 training
---GIT ROOT---
/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer

```

4. user (2026-06-29T17:01:52.882Z)

```

./gs:
./local:
./gcs:
---scheme dirs at root---
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 13:11 gcs:
drwxr-xr-x 4 avidullu avidullu 4096 Jun 25 13:11 gs:
drwxr-xr-x 3 avidullu avidullu 4096 Jun 25 13:11 local:

```

5. assistant (2026-06-29T17:01:55.373Z)

The git root is the badminton-highlight-indexer directory itself. Let me inspect the stray dirs and find the writer code.

6. user (2026-06-29T17:01:56.942Z)

```

===gcs:===
gcs:
gcs:/bucket
gcs:/bucket/videos
gcs:/bucket/videos/x.report.md
gcs:/bucket/videos/x.report.json
gcs:/bucket/videos/x.summary.md

===gs:===
gs:
gs:/b
gs:/bucket
gs:/b/clip.report.json
gs:/b/clip.report.md
gs:/b/clip.summary.md
gs:/bucket/videos
gs:/bucket/videos/cli.report.md
gs:/bucket/videos/x.report.md
gs:/bucket/videos/cli.report.json
gs:/bucket/videos/x.report.json
gs:/bucket/videos/x.summary.md
gs:/bucket/videos/cli.summary.md

===local:===
local:
local:/tmp
local:/tmp/pytest-of-avidullu
local:/tmp/pytest-of-avidullu/pytest-3

```

```

local:/tmp/pytest-of-avidullu/pytest-9
local:/tmp/pytest-of-avidullu/pytest-4
local:/tmp/pytest-of-avidullu/pytest-0
local:/tmp/pytest-of-avidullu/pytest-10
local:/tmp/pytest-of-avidullu/pytest-7
local:/tmp/pytest-of-avidullu/pytest-8
local:/tmp/pytest-of-avidullu/pytest-13
local:/tmp/pytest-of-avidullu/pytest-12
local:/tmp/pytest-of-avidullu/pytest-2
local:/tmp/pytest-of-avidullu/pytest-11
local:/tmp/pytest-of-avidullu/pytest-6
local:/tmp/pytest-of-avidullu/pytest-3/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-2/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-9/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-4/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-0/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-10/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-7/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-8/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-13/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-12/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-11/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-6/test_process_local_scheme_uri_0
local:/tmp/pytest-of-avidullu/pytest-3/test_process_local_scheme_uri_0/ls.report.json
local:/tmp/pytest-of-avidullu/pytest-3/test_process_local_scheme_uri_0/ls.report.md
local:/tmp/pytest-of-avidullu/pytest-3/test_process_local_scheme_uri_0/ls.summary.md
local:/tmp/pytest-of-avidullu/pytest-12/test_process_local_scheme_uri_0/ls.report.json
local:/tmp/pytest-of-avidullu/pytest-12/test_process_local_scheme_uri_0/ls.report.md
local:/tmp/pytest-of-avidullu/pytest-12/test_process_local_scheme_uri_0/ls.summary.md
local:/tmp/pytest-of-avidullu/pytest-2/test_process_local_scheme_uri_0/ls.report.json
local:/tmp/pytest-of-avidullu/pytest-2/test_process_local_scheme_uri_0/ls.report.md
local:/tmp/pytest-of-avidullu/pytest-2/test_process_local_scheme_uri_0/ls.summary.md
local:/tmp/pytest-of-avidullu/pytest-9/test_process_local_scheme_uri_0/ls.report.json
local:/tmp/pytest-of-avidullu/pytest-9/test_process_local_scheme_uri_0/ls.report.md
local:/tmp/pytest-of-avidullu/pytest-9/test_process_local_scheme_uri_0/ls.summary.md
local:/tmp/pytest-of-avidullu/pytest-4/test_process_local_scheme_uri_0/ls.report.json
local:/tmp/pytest-of-avidullu/pytest-4/test_process_local_scheme_uri_0/ls.report.md
local:/tmp/pytest-of-avidullu/pytest-4/test_process_local_scheme_uri_0/ls.summary.md
local:/tmp/pytest-of-avidullu/pytest-0/test_process_local_scheme_uri_0/ls.report.json
local:/tmp/pytest-of-avidullu/pytest-0/test_process_local_scheme_uri_0/ls.report.md
local:/tmp/pytest-of-avidullu/pytest-0/test_process_local_scheme_uri_0/ls.summary.md
local:/tmp/pytest-of-avidullu/pytest-10/test_process_local_scheme_uri_0/ls.report.json
local:/tmp/pytest-of-avidullu/pytest-10/test_process_local_scheme_uri_0/ls.report.md
local:/tmp/pytest-of-avidullu/pytest-10/test_process_local_scheme_uri_0/ls.summary.md
local:/tmp/pytest-of-avidullu/pytest-7/test_process_local_scheme_uri_0/ls.report.json
local:/tmp/pytest-of-avidullu/pytest-7/test_process_local_scheme_uri_0/ls.report.md

```

7. user (2026-06-29T17:01:58.079Z)

```

__init__.py
__pycache__
base.py
gcs.py
gdrive.py
gdrive_api.py
local.py
ref.py
s3.py
---ref.py exists?---
-rw-r--r-- 1 avidullu avidullu 4440 Jun 27 17:27 backend/storage/ref.py

```

8. assistant (2026-06-29T17:02:01.620Z)

The stray dirs contain report/summary files (`.report.md`, `.report.json`, `.summary.md`). The writer is clearly a report-writer that joins paths against a URI string. Let me read `ref.py`

and find the report-writer.

9. user (2026-06-29T17:02:03.047Z)

```
1      """Storage value objects + URI parsing for the pluggable storage layer.
2
3      A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5          <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6          local://<path>             -> local    (local:///abs or local://rel/p)
7          s3://<bucket>/<key>        -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8          gcs://<bucket>/<key>      -> gcs       (gs:// accepted, normalized to gcs://)
9          gdrive://<path>           -> gdrive    (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14
15      from __future__ import annotations
16
17      import re
18      from dataclasses import dataclass
19      from datetime import datetime
20      from typing import Optional
21
22      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
23      _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
24
25
26      @dataclass(frozen=True)
27      class StorageStat:
28          """Lightweight object metadata (uniform across backends)."""
29
30          size: int
31          modified: Optional[datetime] = None
32          etag: Optional[str] = None
33          content_type: Optional[str] = None
34
35
36      @dataclass(frozen=True)
37      class StorageRef:
38          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
39          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
40
41          backend: str
42          bucket: str
43          key: str
44          uri: str
45
46          @property
47          def name(self) -> str:
48              return self.key.rstrip("/").rsplit("/", 1)[-1]
49
50
51      def parse_uri(uri: str) -> StorageRef:
52          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
53
54          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
have no ``//``) is treated as a local filesystem path ? preserving today's
```

```

behaviour.
56     """
57     s = str(uri)
58     m = _SCHEME_RE.match(s)
59     if not m:
60         return StorageRef("local", "", s, f"local://{s}")
61     scheme = m.group(1).lower()
62     rest = s[m.end() :]
63     if scheme == "gs":
64         scheme = "gcs"
65     if scheme in ("local", "gdrive", "gdrive-api"):
66         # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
under a Drive-API
67         # root folder (gdrive-api resolves the path ? a Drive file id).
68         return StorageRef(scheme, "", rest, s)
69     if scheme in _KNOWN:
70         bucket, _, key = rest.partition("/")
71         return StorageRef(scheme, bucket, key, s)
72     raise ValueError(f"unsupported storage URI scheme: {scheme}://{s} (in {s!r})")
73
74
75     def resolve_input_uri(
76         raw: str, *, input_backend: str = "local", input_root: str = ""
77     ) -> str:
78         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
79
80         Precedence (first match wins):
81         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
82         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
83         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
(``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
84         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
``bucket/x`` ?
85         ``gcs://bucket/x``).
86         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
87
88         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
passthrough for every
89         path used today, preserving byte-for-byte behaviour.
90         """
91         s = str(raw)
92         if _SCHEME_RE.match(s): # 1. explicit scheme wins
93             return s
94         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
95         # os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
96         # "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
97         if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
98             return s
99         if input_root: # 3. join a relative name under the configured root
100             return input_root.rstrip("/") + "/" + s.lstrip("/")
101         if (
102             input_backend and input_backend != "local"
103         ): # 4. prefix a bucket/key with the declared backend
104             return f"{input_backend}:{s}"
105         return s # 5. bare/relative local path ? unchanged
106
107

```

```
backend/worker/__main__.py:313:         video_id, segments, status, os.path.join(out_local,
"report.json")
```

11. assistant (2026-06-29T17:02:07.732Z)

The grep pointed to `backend/worker/\_\_main\_\_.py`. Let me look at the worker code around the report-writing logic and trace where `out\_local` comes from.

12. user (2026-06-29T17:02:09.301Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20
21     from __future__ import annotations
22
23     import argparse
24     import csv
25     import json
26     import logging
27     import os
28     import re
29     import sys
30     import tempfile
31     from typing import Any, List
32
33     from backend import storage
34     from backend.compute import ComputeJobSpec, get_compute_backend
35     from backend.config import (
36         StartupCheckError,
37         enforce_startup_checks,
38         load_config,
39         probe_cuda_available,
40     )
41     from backend.infrastructure.database import Database
42     from backend.pipeline.segmenters.base import segmenter_registry
43     from backend.pipeline.validators.base import registry as validator_registry
44     from backend.reporting import emit_indexer_report
45     from backend.results import Failure
46     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
47     from backend.utils.hashing import compute_video_id
48
49     logger = logging.getLogger("backend.worker")
50
51     def _coerce(v: str) -> Any:
52         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
53         str."""
54         low = v.lower()
```

```

51         if low in ("true", "false"):
52             return low == "true"
53         for cast in (int, float):
54             try:
55                 return cast(v)
56             except ValueError:
57                 pass
58         return v
59
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")
68             obj = config
69             for p in parts[:-1]:
70                 obj = getattr(obj, p)
71             setattr(obj, parts[-1], _coerce(val))
72
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76         artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow(
82                     [
83                         f"{float(s.get('start_time', 0.0)):.3f}",
84                         f"{float(s.get('end_time', 0.0)):.3f}",
85                         s.get("shot_count", ""),
86                         s.get("ending_reason", s.get("shot_count_confidence", "")),
87                     ]
88                 )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     "segments": [
101                         {
102                             "start": float(s.get("start_time", 0.0)),
103                             "end": float(s.get("end_time", 0.0)),
104                         }
105                         for s in segments
106                     ],
107                 },
108                 f,
109                 indent=2,
110             )
111
112
113     def _run_startup_checks(config: Any) -> bool:

```

```

114         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
115         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
116         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
117         (returns True, ZERO work) when no deployment.profile is selected.
118
119         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
120         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
121         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
122         no-op of work, not just of output."""
123         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
124         cuda = (
125             probe_cuda_available() if profile else None
126         ) # torch-free on the default-OFF path
127         try:
128             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
129             return True
130         except StartupCheckError:
131             return False # already logged at ERROR by enforce_startup_checks
132
133
134     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
135         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
136         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
137         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
138
139         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
140         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
141         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
142         from backend.infrastructure.execution_policy import PolicyTimeout
143
144         spec = ComputeJobSpec(
145             video_uri=args.video_uri,
146             out_uri=args.out_uri,
147             segmenter=args.segmenter,
148             config_overrides=tuple(args.set or ()),
149         )
150         try:
151             result = compute.run_infer(spec)
152         except PolicyTimeout as e:
153             logger.warning(
154                 "[worker] remote compute did not complete (no marker within budget): %s", e
155             )
156             return 4
157         logger.info(
158             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
159             result.video_id,
160             result.returncode,
161             result.outputs_uri,
162         )
163         return result.returncode
164
165
166     def _write_done_marker(

```



```

167     done_uri: str,
168     video_id: str,
169     returncode: int,
170     segment_count: int,
171     status: str,
172     storage_cfg: dict,
173     scratch: str,
174 ) -> None:
175     """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
176     dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
177     try:
178         marker = {
179             "video_id": video_id,
180             "returncode": returncode,
181             "segment_count": segment_count,
182             "status": status,
183         }
184         local = os.path.join(scratch, "_done.json")
185         with open(local, "w", encoding="utf-8") as f:
186             json.dump(marker, f)
187         storage.put(local, done_uri, config=storage_cfg)
188         logger.info("[worker] wrote completion marker -> %s", done_uri)
189     except Exception as e: # never fail a good run because the marker push hiccuped
190         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
191
192
193 def cmd_infer(args: argparse.Namespace) -> int:
194     config = load_config(args.config) if args.config else load_config()
195     _apply_overrides(config, args.set)
196     if not _run_startup_checks(config):
197         return 2
198
199     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
200     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
201     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
202     # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
203     compute = get_compute_backend(config)
204     if compute is not None:
205         return _dispatch_remote(compute, args)
206
207     storage_cfg = config.storage.model_dump()
208
209     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
210     os.makedirs(scratch, exist_ok=True)
211     config.output.output_dir = scratch
212
213     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
214     local_video = storage.fetch(
215         args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
216     )
217     print(f"[worker] pulled {args.video_uri} -> {local_video}")
218
219     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
220     video_id = compute_video_id(local_video)
221
222     # 3. VALIDATE (same registry as the main CLI).
223     val_results, val_context = validator_registry.run_all_with_context(

```

```

224         local_video, config
225     )
226     failures = [r for r in val_results if not r.passed]
227     db = Database(os.path.join(scratch, config.output.db_name))
228     db.add_video(
229         video_id,
230         local_video,
231         "failed" if failures else "processed",
232         [r.model_dump() for r in val_results],
233     )
234
235     def _fail(rc: int) -> int:
236         # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
237         # dispatcher's bucket-poll terminates FAST + accurately instead of hanging until
238         # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
239         # local run).
240         if getattr(args, "done_uri", None):
241             _write_done_marker(
242                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
243             )
244         return rc
245
246     segments: List[dict] = []
247     segmenter_actual = None
248     segmentation_ran = False
249     status = "failed"
250     try:
251         if failures:
252             print(
253                 "[worker] validation FAILED: "
254                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
255             )
256             return _fail(2)
257         seg_name = args.segmenter or config.indexing.default_segmenter
258         segmenter_cls = segmenter_registry.get(seg_name)
259         if not segmenter_cls:
260             print(
261                 f"[worker] unknown segmenter: {seg_name!r} "
262                 f"(have {list(segmenter_registry.list_available())})"
263             )
264             return _fail(2)
265         segmenter_actual = seg_name
266         result = segmenter_cls(db, config).process_video(
267             video_id, local_video, max_frames=args.max_frames
268         )
269         segmentation_ran = True
270         if isinstance(result, Failure):
271             print(f"[worker] segmenter FAILED: {result.message}")
272             return _fail(3)
273         segments = result.value if hasattr(result, "value") else result
274         status = "success"
275         # Loud, never-silent diagnostic: a 0-segment success is almost always a
276         # empty match ? the cold-start failure mode (the substrate detector yielded no
277         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
278         # for more).
279         if not segments:
280             detector_impl = (
281                 os.environ.get("RALLY_DETECTOR_IMPL")
282                 or getattr(config.indexing, "detector_impl", None)
283                 or "auto"
284             )

```

```

283         logger.warning(
284             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
285             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
286             "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
287             "the WASB env produced no detections, or the input resolution differs
from the tuned "
288             "proxy resolution. Re-run with -v for the native command + frame
counts.",
289             video_id,
290             segmenter_actual,
291             detector_impl,
292         )
293     finally:
294         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
295         emit_indexer_report(
296             db=db,
297             video_id=video_id,
298             video_path=local_video,
299             config=config,
300             val_results=val_results,
301             val_context=val_context,
302             segmenter_requested=args.segmenter,
303             segmenter_actual=segmenter_actual,
304             was_reingest=False,
305             segmentation_ran=segmentation_ran,
306         )
307
308     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
309     out_local = os.path.join(scratch, "outputs", video_id)
310     os.makedirs(out_local, exist_ok=True)
311     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
312     _write_report_json(
313         video_id, segments, status, os.path.join(out_local, "report.json")
314     )
315
316     out_prefix = args.out_uri.rstrip("/")
317     pushed = []
318     for fn in sorted(os.listdir(out_local)):
319         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
320         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
321         pushed.append(uri)
322
323     print(
324         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
325     )
326     for u in pushed:
327         print("    ", u)
328
329     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
330     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
331     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
332     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
333     if getattr(args, "done_uri", None):
334         _write_done_marker(
335             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
336         )

```

```
337         return 0
338
339
340     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
```

13. assistant (2026-06-29T17:02:13.190Z)

The worker writes to a scratch dir, then pushes through the storage layer ? that's clean. But the stray dirs contain ``.report.md``, ``.report.json``, ``.summary.md`` (note the ``.x.report.md`` pattern ? basename + suffix, not ``.report.json`` in a subdir). This is a different writer. Let me find what writes ``.<name>.report.md``/``.summary.md`` and the ``.test_process_local_scheme_uri`` test.

14. user (2026-06-29T17:02:15.105Z)

```
tests/test_motion_segmenter.py:98:def test_process_video_failure_on_unopenable_video():
tests/test_motion_segmenter.py:116:def test_process_video_no_motion_returns_empty_success():
tests/test_motion_segmenter.py:143:def
test_process_video_emits_one_segment_with_honest_metadata():
tests/test_locale_analytics.py:92:def test_process_request_accepts_optional_locale():
tests/test_edge_auth.py:239:def test_process_default_off_tags_no_owner(tmp_path, monkeypatch):
tests/test_edge_auth.py:264:def test_process_edge_auth_missing_token_401(tmp_path, monkeypatch):
tests/test_edge_auth.py:272:def test_process_edge_auth_valid_token_sets_owner(tmp_path,
monkeypatch, keypair):
tests/test_api_endpoints.py:179:def test_process_success_with_mocked_segmenter(env):
tests/test_api_endpoints.py:195:def test_process_records_locale_for_analytics(env):
tests/test_api_endpoints.py:211:def
test_process_validation_failure_preserves_prior_segments(env):
tests/test_api_endpoints.py:227:def test_process_rejects_null_byte_path(env):
tests/test_api_endpoints.py:238:def test_process_uses_configured_output_dir_not_literal_output(
tests/test_api_endpoints.py:655:def test_process_local_input_passes_identity_path(env):
tests/test_api_endpoints.py:668:def test_process_local_missing_file_still_404(env):
tests/test_api_endpoints.py:676:def test_process_nonlocal_uri_skips_exists_and_fetches(env):
tests/test_api_endpoints.py:710:def test_process_bucket_uri_yields_queryable_video_id(env, uri):
tests/test_api_endpoints.py:758:def test_process_unknown_scheme_returns_400(env):
tests/test_api_endpoints.py:767:def test_process_local_scheme_uri_accepted(env):
tests/test_api_endpoints.py:782:def test_process_local_scheme_uri_missing_still_404(env):
tests/test_api_endpoints.py:791:def test_process_hosted_mode_rejects_nonlocal_uri(env):
```

15. user (2026-06-29T17:02:16.374Z)

(Bash completed with no output)

16. assistant (2026-06-29T17:02:21.621Z)

``.test_process_local_scheme_uri_accepted`` is in ``.test_api_endpoints.py``. The report writer isn't in ``.backend/``. Let me search the whole repo for the ``.report.md``/``.summary.md`` writer and look at the API endpoint code.

17. user (2026-06-29T17:02:22.803Z)

(Bash completed with no output)

18. user (2026-06-29T17:02:25.191Z)

```
tests/test_reporting_harvest.py
---FN basename pattern: x.report.md where x is video stem---
```

19. assistant (2026-06-29T17:02:27.586Z)

The ``.<stem>.report.md`` / ``.<stem>.summary.md`` naming convention is in ``.test_reporting_harvest.py``. Let me read that test and find the writer it exercises.

20. user (2026-06-29T17:02:28.070Z)

```
1      """Tests for the indexer observability harvest + footgun rules.
2
3      These exercise the broken-run scenarios the report is meant to make debuggable.
4      obsreport is a soft dependency; skip cleanly if it isn't installed.
5      """
6
7      import os
8
9      import pytest
10
11     pytest.importorskip("obsreport")
12
13     from backend.pipeline.validators.base import ValidationResult # noqa: E402
14     from backend.reporting import emit_indexer_report, harvest, run_signals # noqa: E402
15
16
17     class _FakeDB:
18         def __init__(self, play=None, candidates=None, existing=False):
19             self._play = play or []
20             self._candidates = candidates or []
21             self._existing = existing
22
23         def query_play_segments(self, video_id, filters):
24             return list(self._play)
25
26         def query_candidate_segments(self, video_id, detector=None, only_unlabeled=False):
27             return list(self._candidates)
28
29         def get_video(self, video_id):
30             return {"id": video_id} if self._existing else None
31
32
33     def _vr(name, passed, message="ok"):
34         return ValidationResult(validator_name=name, passed=passed, message=message)
35
36
37     _FFPROBE_OK = {
38         "streams": [
39             {
40                 "width": 1920,
41                 "height": 1080,
42                 "r_frame_rate": "60000/1001",
43                 "codec_name": "hevc",
44             }
45         ],
46         "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "5500000000"},
47     }
48
49
50     def _record(**kw):
51         base = dict(
52             video_id="md5abc",
53             video_path=None,
54             config={
55                 "ai_provider": "gemini",
56                 "ai_model": "gemini-flash",
57                 "indexing": {"default_segmenter": "wasb_hybrid"},
58             },
59             val_results=[_vr("format_check", True), _vr("relevance_check", True)],
60             val_context={"ffprobe_meta": _FFPROBE_OK},
61             play_segments=[
62                 {"duration": 5.0, "metadata": {"detector": "wasb-hybrid-gemini"}}
63             ],
64         )
```

```

64         candidates=[{"id": 1}, {"id": 2}],
65         segmenter_requested="wasb_hybrid",
66         segmenter_actual="wasb_hybrid",
67         was_reingest=False,
68         segmentation_ran=True,
69     )
70     base.update(kw)
71     return harvest.record_run(**base).envelope
72
73
74 def _codes(env):
75     return {f.code for f in env.flags}
76
77
78 def test_healthy_run_is_clean(monkeypatch):
79     monkeypatch.setenv("GEMINI_API_KEY", "x")
80     env = _record()
81     assert env.verdict == "pass"
82     assert _codes(env) == set()
83     assert env.video_meta.fps == 59.94
84     assert env.video_meta.fps_source == "probed"
85     seg = env.get_stage("segmentation")
86     assert {m.key: m.value for m in seg.metrics}["segment_count"] == 1
87     assert {m.key: m.value for m in seg.metrics}["candidate_windows"] == 2
88     assert env.lint() == []
89
90
91 def test_silent_provider_noop_fires_when_no_key_and_zero_segments(monkeypatch):
92     monkeypatch.delenv("GEMINI_API_KEY", raising=False)
93     env = _record(play_segments=[])
94     assert "silent_provider_noop" in _codes(env)
95     assert env.verdict == "fail"
96     flag = next(f for f in env.flags if f.code == "silent_provider_noop")
97     assert flag.faq_ref == "README#Q7"
98
99
100 def test_ai_empty_warns_when_key_present_but_zero_segments(monkeypatch):
101     monkeypatch.setenv("GEMINI_API_KEY", "x")
102     env = _record(play_segments=[])
103     codes = _codes(env)
104     assert "ai_empty_vs_no_rallies" in codes
105     assert "silent_provider_noop" not in codes
106     assert env.verdict == "pass_with_warnings"
107
108
109 def test_oversize_clip_skip_flags_error(monkeypatch):
110     """The provider refusing oversize chunks (4K stream-copy) must surface as an ERROR
111     flag ? the observed live failure was all 7 chunks skipped + '0 segments /
112     success'."""
113     monkeypatch.setenv("GEMINI_API_KEY", "x")
114     env = _record(play_segments=[], run_signals={"oversize_clips_skipped": 7})
115     assert "oversize_clip_skipped" in _codes(env)
116     assert env.verdict == "fail"
117     assert env.get_stage("ai_handoff").details["oversize_clips_skipped"] == 7
118
119
120 def test_no_oversize_signal_records_zero_and_stays_clean(monkeypatch):
121     monkeypatch.setenv("GEMINI_API_KEY", "x")
122     env = _record() # no run_signals at all
123     assert env.get_stage("ai_handoff").details["oversize_clips_skipped"] == 0
124     assert "oversize_clip_skipped" not in _codes(env)
125
126
127 def test_emit_consumes_run_signals_and_flags_report(tmp_path, monkeypatch):
128     """End-to-end: a signal recorded by the segmenter layer is popped by

```

```

128     emit_indexer_report, lands in the written report, and does not leak.""
129     monkeypatch.setenv("GEMINI_API_KEY", "x")
130     video = tmp_path / "match4k.mp4"
131     video.write_bytes(b"x")
132     run_signals.incr("md5sig", "oversize_clips_skipped", 7)
133     db = _FakeDB()
134     paths = emit_indexer_report(
135         db=db,
136         video_id="md5sig",
137         video_path=str(video),
138         config={"ai_provider": "gemini", "indexing": {"default_segementer": "gemini"}},
139         val_results=[_vr("format_check", True)],
140         val_context={"ffprobe_meta": _FFPROBE_OK},
141         segmenter_requested="gemini",
142         segmenter_actual="gemini",
143         was_reingest=False,
144         segmentation_ran=True,
145     )
146     assert paths is not None
147     with open(paths["json"], encoding="utf-8") as f:
148         assert "oversize_clip_skipped" in f.read()
149     # Consumed: a second pop yields nothing (no cross-run leakage).
150     assert run_signals.pop("md5sig") == {}
151
152
153     def test_motion_segementer_flags_synthetic_metadata(monkeypatch):
154         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
155         env = _record(
156             segmenter_requested="motion",
157             segmenter_actual="motion",
158             config={"indexing": {"default_segementer": "motion"}},
159         )
160         codes = _codes(env)
161         assert "synthetic_motion_metadata" in codes
162         # motion is not AI-based -> the provider-noop rule must NOT fire even with no key
163         assert "silent_provider_noop" not in codes
164
165
166     def test_segementer_divergence_fires_only_without_explicit_override(monkeypatch):
167         monkeypatch.setenv("GEMINI_API_KEY", "x")
168         cfg = {"ai_provider": "gemini", "indexing": {"default_segementer": "wasb_hybrid"}}
169         # No explicit request, but a different segmenter ran -> a fallback diverged: flag
170         it.
171         env = _record(segmenter_requested=None, segmenter_actual="gemini", config=cfg)
172         assert "segementer_divergence" in _codes(env)
173         # Explicit --segmenter override that differs from config default is NOT a
174         divergence.
175         env2 = _record(segmenter_requested="gemini", segmenter_actual="gemini", config=cfg)
176         assert "segementer_divergence" not in _codes(env2)
177
178
179     def test_reingest_info_flag(monkeypatch):
180         monkeypatch.setenv("GEMINI_API_KEY", "x")
181         env = _record(was_reingest=True)
182         assert "reingest_replaced_prior" in _codes(env)
183
184
185     def test_validation_failure_halts_and_flags(monkeypatch):
186         monkeypatch.setenv("GEMINI_API_KEY", "x")
187         env = _record(
188             val_results=[
189                 _vr("format_check", True),
190                 _vr("relevance_check", False, "court color not found"),
191             ],
192             play_segments=[],
193         )

```

```

191         segmenter_actual=None,
192         segmentation_ran=False,
193     )
194     assert env.get_stage("validation").status == "error"
195     assert env.get_stage("segmentation").status == "not_run"
196     assert "validation_failed" in _codes(env)
197     assert env.verdict == "fail"
198
199
200 def test_fps_fallback_flag_when_no_probe_but_segmented(monkeypatch):
201     monkeypatch.setenv("GEMINI_API_KEY", "x")
202     env = _record(val_context={}, segmentation_ran=True)
203     assert env.video_meta.fps_source == "fallback"
204     assert "fps_fallback_used" in _codes(env)
205
206
207 def test_emit_writes_files_next_to_video(tmp_path, monkeypatch):
208     monkeypatch.setenv("GEMINI_API_KEY", "x")
209     video = tmp_path / "match.mp4"
210     video.write_bytes(b"not really a video")
211     db = _FakeDB(
212         play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
213         candidates=[{"id": 1}],
214     )
215     paths = emit_indexer_report(
216         db=db,
217         video_id="md5abc",
218         video_path=str(video),
219         config={
220             "ai_provider": "gemini",
221             "indexing": {"default_segmenter": "wasb_hybrid"},
222         },
223         val_results=[_vr("format_check", True)],
224         val_context={"ffprobe_meta": _FFPROBE_OK},
225         segmenter_requested="wasb_hybrid",
226         segmenter_actual="wasb_hybrid",
227         was_reingest=False,
228         segmentation_ran=True,
229     )
230     assert paths is not None
231     assert os.path.exists(paths["json"])
232     assert (tmp_path / "match.report.md").exists()
233     assert (tmp_path / "match.summary.md").exists()
234
235
236 def test_emit_accepts_typed_appconfig(tmp_path, monkeypatch):
237     """Reporting must work whether main passes a plain dict or the typed AppConfig
238     (backend.config) ? the reporting layer normalizes AppConfig -> dict."""
239     from backend.config import AppConfig
240
241     monkeypatch.setenv("GEMINI_API_KEY", "x")
242     video = tmp_path / "m.mp4"
243     video.write_bytes(b"x")
244     db = _FakeDB(
245         play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
246         candidates=[{"id": 1}],
247     )
248     cfg = AppConfig() # typed config object, not a dict
249     paths = emit_indexer_report(
250         db=db,
251         video_id="md5cfg",
252         video_path=str(video),
253         config=cfg,
254         val_results=[_vr("format_check", True)],
255         val_context={"ffprobe_meta": _FFPROBE_OK},

```



```

256         segmenter_requested="wasb_hybrid",
257         segmenter_actual="wasb_hybrid",
258         was_reingest=False,
259         segmentation_ran=True,
260     )
261     assert paths is not None
262     assert os.path.exists(paths["json"])
263
264
265     def test_emit_disabled_via_config(tmp_path, monkeypatch):
266         monkeypatch.setenv("GEMINI_API_KEY", "x")
267         db = _FakeDB()
268         paths = emit_indexer_report(
269             db=db,
270             video_id="md5abc",
271             video_path=str(tmp_path / "m.mp4"),
272             config={"report": {"enabled": False}},
273             val_results=[_vr("format_check", True)],
274             val_context={},
275             segmenter_actual="motion",
276             segmentation_ran=True,
277         )
278         assert paths is None
279

```

21. assistant (2026-06-29T17:02:31.464Z)

Confirmed. The writer writes files "next to the video" ? ``<video_stem>.report.md``, ``<video_stem>.summary.md``, ``<video_stem>.report.json``. When ``video_path`` is a URI like ``gcs://bucket/videos/x.mp4``, deriving the report path next to the video produces ``gcs://bucket/videos/x.report.md`` (because ``os.path`` collapses ``///`` ? ````). Let me find ``emit_indexer_report`` in ``backend/reporting``.

22. user (2026-06-29T17:02:33.860Z)

```

total 44
drwxr-xr-x  3 avidullu avidullu  4096 Jun 27 17:27 .
drwxr-xr-x 20 avidullu avidullu  4096 Jun 27 17:27 ..
-rw-r--r--  1 avidullu avidullu  3050 Jun 27 17:27 __init__.py
drwxr-xr-x  2 avidullu avidullu  4096 Jun 29 11:11 __pycache__
-rw-r--r--  1 avidullu avidullu 10389 Jun 27 17:27 harvest.py
-rw-r--r--  1 avidullu avidullu  1259 Jun 27 17:27 run_signals.py
-rw-r--r--  1 avidullu avidullu  8908 Jun 25 11:46 spec.yaml
---
backend/reporting/__init__.py:29: def emit_indexer_report(
backend/reporting/__init__.py:33:     video_path: Optional[str],
backend/reporting/__init__.py:62:         video_path=video_path,
backend/reporting/harvest.py:33:     val_context: Optional[Dict[str, Any]], video_path:
Optional[str]
backend/reporting/harvest.py:38:     if video_path and os.path.exists(video_path):
backend/reporting/harvest.py:40:         meta["size_mb"] = round(os.path.getsize(video_path)
/ (1024 * 1024), 1)
backend/reporting/harvest.py:206:     video_path: Optional[str],
backend/reporting/harvest.py:227:         video_path=video_path,
backend/reporting/harvest.py:231:     vm = video_meta_from_context(val_context, video_path)

```

23. user (2026-06-29T17:02:36.854Z)

```

1     """Per-video observability reports for the indexer.
2
3     `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4     no-op and the pipeline runs exactly as before. The single entrypoint
5     `emit_indexer_report(...)` is designed to be called from a `finally` block and
6     NEVER raises ? emitting a report must not be able to break a processing run.

```

```

7     """
8
9     from __future__ import annotations
10
11    from typing import Any, Dict, List, Optional
12
13    from . import run_signals # no obsreport dependency ? always importable
14
15    try: # soft dependency ? never hard-fail the pipeline on a reporting import
16        import obsreport # noqa: F401
17
18        from . import harvest
19
20        OBSREPORT_AVAILABLE = True
21    except Exception: # pragma: no cover - exercised only when obsreport is absent
22        OBSREPORT_AVAILABLE = False
23
24
25    def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
26        return bool((config or {}).get("report", {}).get("enabled", True))
27
28
29    def emit_indexer_report(
30        *,
31        db,
32        video_id: str,
33        video_path: Optional[str],
34        config: Optional[Dict[str, Any]],
35        val_results: List[Any],
36        val_context: Optional[Dict[str, Any]] = None,
37        segmenter_requested: Optional[str] = None,
38        segmenter_actual: Optional[str] = None,
39        was_reingest: bool = False,
40        segmentation_ran: bool = True,
41    ) -> Optional[Dict[str, str]]:
42        """Harvest + write the report. Returns the written file paths, or None if
43        reporting is unavailable/disabled/failed. Never raises."""
44        config = config or {}
45        # Accept either a plain dict or a typed config object (e.g.
46        backend.config.AppConfig).
47        # Normalize to a plain nested dict so harvest's dict access works uniformly.
48        if hasattr(config, "model_dump"):
49            try:
50                config = config.model_dump(mode="json")
51            except Exception:
52                config = {}
53        # Consume run signals unconditionally (even when reporting is off) so the
54        # process-local store never accumulates entries for finished runs.
55        signals = run_signals.pop(video_id)
56        if not OBSREPORT_AVAILABLE or not report_enabled(config):
57            return None
58        try:
59            play_segments = db.query_play_segments(video_id, {}) if db is not None else []
60            candidates = db.query_candidate_segments(video_id) if db is not None else []
61            rec = harvest.record_run(
62                video_id=video_id,
63                video_path=video_path,
64                config=config,
65                val_results=val_results,
66                val_context=val_context,
67                play_segments=play_segments,
68                candidates=candidates,
69                segmenter_requested=segmenter_requested,
70                segmenter_actual=segmenter_actual,
71                was_reingest=was_reingest,

```

```

71         segmentation_ran=segmentation_ran,
72         run_signals=signals,
73     )
74     output_dir = (config.get("report") or {}).get("output_dir")
75     res = rec.write(output_dir=output_dir)
76     return res.paths or None
77 except Exception: # pragma: no cover - defensive; reporting must never break a run
78     return None
79

```

24. assistant (2026-06-29T17:02:39.580Z)

Now I need to see `rec.write(output\_dir=output\_dir)` ? this is where paths are derived. The `record\_run` returns a `rec` object; let me look at `harvest.py` and the `record\_run`/`write` logic.

25. user (2026-06-29T17:02:40.995Z)

```

1     """Build an observability report for one indexer run.
2
3     Pure-ish harvest: given the validation results/context, the persisted segments,
4     and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5     *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6     so this module only records facts faithfully ? it does not decide warnings.
7
8     Imported only after `backend.reporting` confirms obsreport is installed, so it may
9     import obsreport freely.
10    """
11
12    from __future__ import annotations
13
14    import os
15    from pathlib import Path
16    from typing import Any, Dict, List, Optional
17
18    try:
19        import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
20    except ImportError:
21        obsreport = None # type: ignore[assignment]
22
23    _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
24    _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
25
26
27    def load_spec() -> "obsreport.ReportSpec":
28        return obsreport.load_spec(_SPEC_PATH)
29
30
31    # --- media metadata -----
32    def video_meta_from_context(
33        val_context: Optional[Dict[str, Any]], video_path: Optional[str]
34    ) -> Dict[str, Any]:
35        """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
36        distinguishes a real probe from the absence of one (the fallback footgun)."""
37        meta: Dict[str, Any] = {"fps_source": "unknown"}
38        if video_path and os.path.exists(video_path):
39            try:
40                meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
41            except OSError:
42                pass
43
44        ffmata = (val_context or {}).get("ffprobe_meta") or {}
45        streams = ffmata.get("streams") or []
46        fmt = ffmata.get("format") or {}
47        if streams:

```

```

48         s = streams[0]
49         w, h = s.get("width"), s.get("height")
50         meta["width"], meta["height"] = w, h
51         if w and h:
52             meta["resolution"] = f"{w}x{h}"
53         fps_str = s.get("r_frame_rate", "0/1")
54         try:
55             num, den = (int(x) for x in fps_str.split("/"))
56             if den:
57                 meta["fps"] = round(num / den, 2)
58                 meta["fps_source"] = "probed"
59         except (ValueError, ZeroDivisionError):
60             pass
61         if s.get("codec_name"):
62             meta["container"] = fmt.get("format_name") or s.get("codec_name")
63     if fmt.get("duration"):
64         try:
65             meta["duration_s"] = round(float(fmt["duration"]), 1)
66         except (ValueError, TypeError):
67             pass
68     if fmt.get("format_name"):
69         meta["container"] = fmt.get("format_name")
70
71     # Audio readiness signal (for Input Quality UX extension per review item 2).
72     # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
73     # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
74     # We only record presence here (cheap, from ffprobe already in context); the actual
75     # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
76     # tool for now.
77     audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
78     meta["audio_present"] = len(audio_streams) > 0
79     return meta
80
81
82     # --- stage builders -----
83     def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
84         with rec.stage("validation") as st:
85             total = len(val_results)
86             passed = sum(1 for r in val_results if getattr(r, "passed", False))
87             st.add_metric("checks_passed", passed)
88             st.add_metric("checks_total", total)
89             failures = [r for r in val_results if not getattr(r, "passed", True)]
90             for r in failures:
91                 st.messages.append(
92                     f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r, 'message',
93 ''}})"
94                 )
95             st.status = "error" if failures else ("ok" if total else "not_run")
96
97     def _segmentation_stage(
98         rec: "obsreport.RunRecorder",
99         play_segments: List[Dict[str, Any]],
100         candidates: List[Dict[str, Any]],
101         segmenter_requested: Optional[str],
102         segmenter_actual: Optional[str],
103         config_default: Optional[str],
104         was_reingest: bool,
105         ran: bool,
106     ) -> None:
107         with rec.stage("segmentation") as st:
108             if not ran:
109                 st.status = "not_run"
110             seg_count = len(play_segments)

```

```

111     total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
112     detector_tag = None
113     if play_segments:
114         detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
115         st.add_metric("candidate_windows", len(candidates))
116         st.add_metric("segment_count", seg_count, audience="both")
117         st.add_metric("total_play_s", total_play, unit="s")
118         # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
119         # pick it, and it differs from config's default (the genuine surprise case).
120         # An explicit --segmenter override or a halted run is not a divergence.
121         matches_default = True
122         if ran and segmenter_actual and config_default:
123             matches_default = segmenter_actual == config_default
124         st.details.update(
125             {
126                 "segmenter_requested": segmenter_requested,
127                 "segmenter_actual": segmenter_actual,
128                 "config_default": config_default,
129                 "segmenter_explicitly_requested": segmenter_requested is not None,
130                 "matches_config_default": matches_default,
131                 "was_reingest": was_reingest,
132                 "detector": detector_tag,
133             }
134         )
135         # Surface metadata trustworthiness at the data level (not just in a warning):
136         # the motion baseline fabricates per-segment metadata; everything else measures
137         it.
138         if ran and segmenter_actual:
139             st.details["metadata_source"] = (
140                 "synthetic/heuristic ? NOT measured"
141                 if segmenter_actual == "motion"
142                 else "measured (detector/AI)"
143             )
144
145     def _ai_stage(
146         rec: "obsreport.RunRecorder",
147         config: Dict[str, Any],
148         segmenter_actual: Optional[str],
149         play_segments: List[Dict[str, Any]],
150         ran: bool,
151         run_signals: Optional[Dict[str, Any]] = None,
152     ) -> None:
153         """Only meaningful for AI-based segmenters. Records whether the provider key was
154         present ? the signal the silent_provider_noop rule keys on ? plus any in-process
155         run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
156         ai_based = segmenter_actual in _AI_SEGMENTERS
157         provider = config.get("ai_provider") or "gemini"
158         key_env = f"{provider.upper()}_API_KEY"
159         with rec.stage("ai_handoff") as st:
160             if not ran or not ai_based:
161                 st.status = "not_run" if not ran else "skipped"
162             st.details.update(
163                 {
164                     "ai_based": ai_based,
165                     "provider": provider if ai_based else None,
166                     "model": config.get("ai_model") if ai_based else None,
167                     "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
168                     # Chunks the provider refused as over its upload cap (no analysis ran
169                     # for
170                     # those time ranges). Detail name couples to the oversize_clip_skipped
171                     "oversize_clips_skipped": int(
172                         (run_signals or {}).get("oversize_clips_skipped", 0)
173                     )
174                 }
175             )

```

```

173             if ai_based
174             else None,
175         }
176     )
177     if ai_based:
178         st.add_metric("confirmed_segments", len(play_segments))
179
180
181 def _highlights(
182     rec: "obsreport.RunRecorder",
183     play_segments: List[Dict[str, Any]],
184     video_meta: Dict[str, Any],
185 ) -> None:
186     total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
187     coverage = None
188     dur = video_meta.get("duration_s")
189     if dur:
190         coverage = round(100.0 * total / dur, 1)
191     notes = []
192     if not play_segments:
193         notes.append("No highlights were produced for this video.")
194     rec.set_highlights(
195         segment_count=len(play_segments),
196         total_highlight_s=total,
197         coverage_pct=coverage,
198         notes=notes,
199     )
200
201
202 # --- top-level -----
203 def record_run(
204     *,
205     video_id: str,
206     video_path: Optional[str],
207     config: Dict[str, Any],
208     val_results: List[Any],
209     val_context: Optional[Dict[str, Any]],
210     play_segments: List[Dict[str, Any]],
211     candidates: List[Dict[str, Any]],
212     segmenter_requested: Optional[str] = None,
213     segmenter_actual: Optional[str] = None,
214     was_reingest: bool = False,
215     segmentation_ran: bool = True,
216     tool_version: Optional[str] = None,
217     spec: Optional["obsreport.ReportSpec"] = None,
218     run_signals: Optional[Dict[str, Any]] = None,
219     **recorder_kwargs: Any,
220 ) -> "obsreport.RunRecorder":
221     """Assemble a finalized recorder for an indexer run (does NOT write)."""
222     rec = obsreport.RunRecorder(
223         "badminton-highlight-indexer",
224         tool_version=tool_version,
225         video_id=video_id,
226         video_id_kind="file_md5",
227         video_path=video_path,
228         spec=spec if spec is not None else load_spec(),
229         **recorder_kwargs,
230     )
231     vm = video_meta_from_context(val_context, video_path)
232     # If we never probed an fps but segmentation still ran, the segmenter must have
233     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
234     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?"
235     (fallback)".
236     if vm.get("fps_source") == "unknown" and segmentation_ran:
237         vm["fps_source"] = "fallback"

```

```

237         if vm.get("fps") is None:
238             vm["fps"] = (
239                 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0 / 1280)
240             )
241         rec.set_video_meta(**vm)
242
243         _validation_stage(rec, val_results)
244         config_default = (config.get("indexing") or {}).get("default_segmenter")
245         _segmentation_stage(
246             rec,
247             play_segments,
248             candidates,
249             segmenter_requested,
250             segmenter_actual,
251             config_default,
252             was_reingest,
253             segmentation_ran,
254         )
255         _ai_stage(
256             rec, config, segmenter_actual, play_segments, segmentation_ran, run_signals
257         )
258         _highlights(rec, play_segments, vm)
259
260         rec.finalize()
261         return rec
262

```

26. assistant (2026-06-29T17:02:44.601Z)

The path derivation is inside ``obsreport.RunRecorder.write()`` ? ``obsreport`` is an external library. The ``RunRecorder`` is given ``video_path`` and, when ``write(output_dir=None)``, derives ``<stem>.report.md`` next to the video. Let me confirm where `obsreport` lives and how ``write()`` derives the path.

27. user (2026-06-29T17:02:48.599Z)

```

/home/avidullu/projects/khelsutra-guru/sports-obsreport/src/obsreport
---
['Clause', 'Flag', 'HighlightSummary', 'Identity', 'Metric', 'ReportEnvelope', 'ReportSpec',
'Rule', 'RunRecorder', 'SCHEMA_VERSION', 'Section', 'Stage', 'VideoMeta', 'WriteResult',
'apply_rules', 'diff', 'envelope', 'io', 'load_spec', 'load_spec_dict', 'load_spec_text',
'recorder', 'render', 'render_customer', 'render_technical', 'report_paths', 'rules',
'run_spec', 'spec', 'stable_dict', 'stable_json', 'to_json', 'verdict_from_flags', 'version',
'write_report']

```

28. assistant (2026-06-29T17:02:51.078Z)

``obsreport`` is a sibling repo. There's a ``report_paths`` function and ``RunRecorder.write``. Let me look at how ``write`/`report_paths`` derive the output path from ``video_path``.

29. user (2026-06-29T17:02:55.321Z)

```

recorder.py:37:         video_path: Optional[str] = None,
recorder.py:54:         video_path=video_path,
recorder.py:132:     def write(self, **io_kwargs: Any) -> WriteResult:
recorder.py:136:         video_path=self.envelope.identity.video_path,
io.py:4:     /videos/match.mp4 -> /videos/match.report.json      (machine, source of truth)
io.py:5:                     /videos/match.report.md            (technical, debuggable)
io.py:6:                     /videos/match.summary.md           (customer-friendly)
io.py:8:Overridable with ``output_dir``. For remote/no-local-file runs (e.g. the
io.py:9:collector before download), output falls back to ``output_dir`` or
io.py:35:def _base(video_path: Optional[str], output_dir: Optional[str], video_id:
Optional[str], stem: Optional[str]):
io.py:39:     elif video_path:

```

```

io.py:40:         name = os.path.splitext(os.path.basename(video_path))[0]
io.py:46:     if output_dir:
io.py:47:         return output_dir, name
io.py:48:     if video_path:
io.py:49:         return os.path.dirname(os.path.abspath(video_path)) or ".", name
io.py:51:     return os.path.join("reports", video_id or "run"), name
io.py:54: def report_paths(
io.py:55:     video_path: Optional[str] = None,
io.py:56:     output_dir: Optional[str] = None,
io.py:61:     directory, name = _base(video_path, output_dir, video_id, stem)
io.py:63:     "json": os.path.join(directory, f"{name}.report.json"),
io.py:64:     "technical_md": os.path.join(directory, f"{name}.report.md"),
io.py:65:     "customer_md": os.path.join(directory, f"{name}.summary.md"),
io.py:71:     directory = os.path.dirname(path) or "."
io.py:79: def write_report(
io.py:83:     video_path: Optional[str] = None,
io.py:84:     output_dir: Optional[str] = None,
io.py:93:         video_path=video_path,
io.py:94:         output_dir=output_dir,

```

30. assistant (2026-06-29T17:02:57.223Z)

Found the root cause. Let me read the exact lines in `io.py`.

31. user (2026-06-29T17:02:58.044Z)

```

1     """File output: the path convention + safe writes.
2
3     Convention (next to the video by default):
4         /videos/match.mp4 -> /videos/match.report.json      (machine, source of truth)
5                             /videos/match.report.md         (technical, debuggable)
6                             /videos/match.summary.md         (customer-friendly)
7
8     Overridable with ``output_dir``. For remote/no-local-file runs (e.g. the
9     collector before download), output falls back to ``output_dir`` or
10    ``reports/<video_id>/``.
11
12    THE CARDINAL RULE: writing a report must NEVER break the host pipeline run.
13    Every write is wrapped; failures are collected into a ``WriteResult`` and
14    returned, never raised.
15    """
16    from __future__ import annotations
17
18    import os
19    from dataclasses import dataclass, field
20    from typing import Dict, List, Optional
21
22    from .envelope import ReportEnvelope
23    from .render.json_render import to_json
24    from .render.markdown import render_customer, render_technical
25    from .spec import ReportSpec
26
27
28    @dataclass
29    class WriteResult:
30        ok: bool
31        paths: Dict[str, str] = field(default_factory=dict)
32        errors: List[str] = field(default_factory=list)
33
34
35    def _base(video_path: Optional[str], output_dir: Optional[str], video_id: Optional[str],
36    stem: Optional[str]):
37        """Return (directory, basename) for the report files."""
38        if stem:
39            name = stem

```



```

39     elif video_path:
40         name = os.path.splitext(os.path.basename(video_path))[0]
41     elif video_id:
42         name = video_id
43     else:
44         name = "report"
45
46     if output_dir:
47         return output_dir, name
48     if video_path:
49         return os.path.dirname(os.path.abspath(video_path)) or ".", name
50     # remote / no local file
51     return os.path.join("reports", video_id or "run"), name
52
53
54 def report_paths(
55     video_path: Optional[str] = None,
56     output_dir: Optional[str] = None,
57     video_id: Optional[str] = None,
58     stem: Optional[str] = None,
59 ) -> Dict[str, str]:
60     """Resolve the three report file paths (no IO)."""
61     directory, name = _base(video_path, output_dir, video_id, stem)
62     return {
63         "json": os.path.join(directory, f"{name}.report.json"),
64         "technical_md": os.path.join(directory, f"{name}.report.md"),
65         "customer_md": os.path.join(directory, f"{name}.summary.md"),
66     }
67
68
69 def _write_text(path: str, text: str) -> None:
70     """Atomic write: temp file in the same dir, then os.replace. Patchable in tests."""
71     directory = os.path.dirname(path) or "."
72     os.makedirs(directory, exist_ok=True)
73     tmp = f"{path}.tmp"
74     with open(tmp, "w", encoding="utf-8", newline="\n") as f:
75         f.write(text)
76     os.replace(tmp, path)
77
78
79 def write_report(
80     envelope: ReportEnvelope,
81     spec: Optional[ReportSpec] = None,
82     *,
83     video_path: Optional[str] = None,
84     output_dir: Optional[str] = None,
85     video_id: Optional[str] = None,
86     stem: Optional[str] = None,
87     write_customer: bool = True,
88 ) -> WriteResult:
89     """Render + write json / technical md / (optional) customer md. Never raises."""
90     result = WriteResult(ok=True)
91     try:
92         paths = report_paths(
93             video_path=video_path,
94             output_dir=output_dir,
95             video_id=video_id or envelope.identity.video_id,
96             stem=stem,
97         )
98     except Exception as e: # pragma: no cover - defensive
99         return WriteResult(ok=False, errors=[f"path resolution failed: {e}"])
100
101     jobs = [
102         ("json", paths["json"], lambda: to_json(envelope)),
103         ("technical_md", paths["technical_md"], lambda: render_technical(envelope,
104 spec))]

```

```

103         if write_customer:
104             jobs.append(("customer_md", paths["customer_md"], lambda:
render_customer(envelope, spec)))
105
106         for key, path, render in jobs:
107             try:
108                 _write_text(path, render())
109                 result.paths[key] = path
110             except Exception as e:

```

32. assistant (2026-06-29T17:03:02.880Z)

Now I fully understand the bug. The root cause is `io.py:48-49` in obsreport:

```

```python
if video_path:
 return os.path.dirname(os.path.abspath(video_path)) or ".", name
...

```

When `video\_path = "gcs://bucket/videos/x.mp4"`:

- `os.path.abspath("gcs://bucket/videos/x.mp4")` collapses the `///` and treats it as a relative path ? ``<cwd>/gcs://bucket/videos/x.mp4``
- `os.path.dirname(...)` ? ``<cwd>/gcs://bucket/videos``
- Then `\_write\_text` does `os.makedirs(directory)` ? creates `gcs://bucket/videos/` under the repo root (cwd).

Let me verify this empirically and confirm `\_base`'s abspath behavior with a scheme URI.

### 33. user (2026-06-29T17:03:07.695Z)

```

'gcs://bucket/videos/x.mp4' -> dir= '/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/gcs://bucket/videos' name= 'x'
'gs://b/clip.mp4' -> dir= '/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/gs://b' name= 'clip'
'local:///tmp/pytest-of-avidullu/pytest-3/test_process_local_scheme_uri_0/ls.mp4' -> dir= '/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/local:///tmp/pytest-of-avidullu/pytest-3/test_process_local_scheme_uri_0' name= 'ls'
'local:///tmp/x/ls.mp4' -> dir= '/home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer/local:///tmp/x' name= 'ls'

```

### 34. assistant (2026-06-29T17:03:11.809Z)

Exactly reproduced. The dirs `gcs://bucket/videos`, `gs://b`, `local:///tmp/...` are created under the repo root (cwd). Now let me find WHO calls `emit\_indexer\_report` with a URI as `video\_path`. The `gs://bucket/videos/cli.\*` artifacts (note "cli") suggest the CLI/worker path. Let me find all callers.

### 35. user (2026-06-29T17:03:15.615Z)

```

backend/worker/__main__.py:40:from backend.reporting import emit_indexer_report
backend/worker/__main__.py:295: emit_indexer_report(
backend/worker/__main__.py:298: video_path=local_video,
backend/pipeline/detectors/base.py:59:segmenter Results (or pass detector_health into
emit_indexer_report context), the
backend/api/job_worker.py:78: video_path=job["video_path"],
backend/eval/served_gate_a.py:88: cw, points, fps, frame_width, video_path="",
idx_cfg=idx_cfg_typed
backend/main.py:68:from backend.reporting import emit_indexer_report
backend/main.py:950: paths = emit_indexer_report(
backend/main.py:953: video_path=req.video_path,
backend/main.py:1722: paths = emit_indexer_report(
backend/main.py:1725: video_path=args.video_path,
=== test callers that pass scheme URI as video_path ===
tests/test_worker.py:156: monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)

```

```
tests/test_worker.py:221: monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
tests/test_worker.py:264: monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
```

### 36. assistant (2026-06-29T17:03:18.961Z)

Two API/CLI callers in `backend/main.py` pass `req.video\_path` / `args.video\_path` directly ? that's where a `gcs:///`/`gs:///`/`local:///` URI flows in unmodified. The worker (`\_\_main\_\_.py:298`) passes `local\_video` (already fetched to scratch), so it's safe. Let me examine `main.py` around the two call sites and the `job\_worker.py` caller.

### 37. user (2026-06-29T17:03:19.941Z)

```
920 "results": [r.model_dump() for r in val_results],
921 "play_segments": segments,
922 }
923 return response
924 finally:
925 # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
926 # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
927 # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
928 # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
929 if isinstance(response, dict):
930 response.setdefault(
931 "compute",
932 {
933 "path": compute_actual or "not_run",
934 "requested": getattr(req, "compute", None),
935 },
936)
937 prov = response.get("compute", {}) if isinstance(response, dict) else {}
938 logger.info(
939 "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
940 video_id,
941 getattr(req, "compute", None),
942 prov.get("path"),
943 prov.get("dispatched"),
944 (response or {}).get("status") if isinstance(response, dict) else None,
945)
946 # Always emit the per-video observability report (next to the video, or to
947 # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
948 # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
949 # doesn't surface arbitrary run signals.)
950 paths = emit_indexer_report(
951 db=db,
952 video_id=video_id,
953 video_path=req.video_path,
954 config=config,
955 val_results=val_results,
956 val_context=val_context,
957 segmenter_requested=req.segmenter,
958 segmenter_actual=segmenter_actual,
959 was_reingest=was_reingest,
960 segmentation_ran=segmentation_ran,
961)
962 if paths and isinstance(response, dict):
963 response["reports"] = paths
964
965
966 @app.post("/api/process", status_code=202)
```

```

967 def process_video(req: ProcessRequest, request: Request):
968 """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
969
970 Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
971 client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
972 runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
973 worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
974 """
975 # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
976 # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
977 # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
978 try:
979 owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)

```

### 38. user (2026-06-29T17:03:20.672Z)

```

1695 segmenter_cls = segmenter_registry.get(seg_name)
1696 if not segmenter_cls:
1697 print(f"[FAIL] Segmenter '{seg_name}' not found.")
1698 return
1699
1700 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1701 segmenter_actual = seg_name
1702 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1703 segmenter = segmenter_cls(db_instance, global_config)
1704 result = segmenter.process_video(
1705 video_id, local_video, max_frames=args.max_frames
1706)
1707 segmentation_ran = True
1708
1709 if isinstance(result, Failure):
1710 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1711 print("[FAIL] Indexing halted for video.")
1712 segments = []
1713 else:
1714 segments = result.value if hasattr(result, "value") else result
1715 print(
1716 f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}"
1717)
1718 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1719 _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1720 finally:
1721 # Always emit the per-video observability report. Never raises.
1722 paths = emit_indexer_report(
1723 db=db_instance,
1724 video_id=video_id,
1725 video_path=args.video_path,
1726 config=global_config,
1727 val_results=val_results,
1728 val_context=val_context,
1729 segmenter_requested=args.segmenter,
1730 segmenter_actual=segmenter_actual,
1731 was_reingest=was_reingest,
1732 segmentation_ran=segmentation_ran,
1733)
1734 if paths:
1735 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1736 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1737 print(f"[REPORT] Machine JSON : {paths.get('json')}")
1738 return

```

**39. assistant (2026-06-29T17:03:27.252Z)**

Now let me check the API test that produces the `gs:/bucket/videos/cli.\*` and confirm `req.video\_path` carries the raw URI. Let me look at `test\_process\_local\_scheme\_uri\_accepted` and the bucket-uri test in `test\_api\_endpoints.py`, and check whether `local\_video` (the fetched/resolved path) is available at these call sites as a safer value.

**40. user (2026-06-29T17:03:28.495Z)**

```

655 def test_process_local_input_passes_identity_path(env):
656 """Default (local) input must reach the segmenter as the EXACT same path ? identity,
no copy
657 (the M2 zero-behaviour-change guarantee for today's local workflow)."""
658 client, db, tmp_path, mp = env
659 vid = _video(tmp_path)
660 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
661 mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
662 _CaptureSeg.last_path = None
663 r = client.post("/api/process", json={"video_path": str(vid), "segmenter": "x"})
664 assert r.status_code == 202
665 assert _CaptureSeg.last_path == str(vid)
666
667
668 def test_process_local_missing_file_still_404(env):
669 """A LOCAL input that doesn't exist still fast-fails 404 ? the existence gate is
preserved
670 for local paths (only non-local URIs skip it)."""
671 client = env[0]
672 r = client.post("/api/process", json={"video_path": "/no/such/local/file.mp4"})
673 assert r.status_code == 404
674
675
676 def test_process_nonlocal_uri_skips_exists_and_fetches(env):
677 """A gs:// input is NOT 404'd at submit (can't os.path.exists a bucket object); the
worker
678 fetches it to a local scratch file and the segmenter sees that LOCAL path, not the
URI."""
679 client, db, tmp_path, mp = env
680
681 def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
682 with open(dst, "wb") as fh:
683 fh.write(b"fetched-video-bytes")
684 return dst
685
686 mp.setattr(m.storage, "fetch", fake_fetch)
687 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
688 mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
689 _CaptureSeg.last_path = None
690 r = client.post(
691 "/api/process",
692 json={"video_path": "gs://bucket/videos/x.mp4", "segmenter": "x"},
693)
694 assert r.status_code == 202 # NOT 404 ? non-local skips the exists gate
695 job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
696 assert job["status"] == "done"
697 assert _CaptureSeg.last_path is not None
698 assert not _CaptureSeg.last_path.startswith(
699 "gs://"
700) # a fetched LOCAL path, not the URI
701 assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"
702 # Provenance: the DB record keeps the SOURCE URI (not the ephemeral scratch path).
703 rec = db.get_video(job["video_id"])
704 assert rec is not None and rec["filepath"] == "gs://bucket/videos/x.mp4"

```

```

705
706
707 @pytest.mark.parametrize(
708 "uri", ["gs://bucket/videos/x.mp4", "gcs://bucket/videos/x.mp4"]
709)
710 def test_process_bucket_uri_yields_queryable_video_id(env, uri):
711 """B-P1 lock (default single-user mode): a bucket URI ? both the `gs://` alias AND
the canonical
712 `gcs://` scheme ? posted to /api/process produces a video_id the SPA back-half can
LOAD, i.e.
713 POST /api/query returns the rallies. Pins the two facts the older gs:// test left
open:
714 1. the video_id is the MD5 of the FETCHED bytes, NOT a hash of the URI string, and
715 2. the play_segments are queryable (the SPA's POST /api/query contract).
716 Counterweight = test_process_hosted_mode_rejects_nonlocal_uri (gs:// -> 400 when
allowed_video_root
717 is set). NOTE: the m.storage.fetch fake proves the CODE routing, not the live GCS
download (that
718 needs google-cloud-storage + ADC on the box ? verified at deploy time, not in this
offline suite)."""
719 client, db, tmp_path, mp = env
720 payload = b"bucket-fetched-bytes-payload"
721
722 def fake_fetch(uri_, dst=None, *, config=None, overwrite=False):
723 with open(dst, "wb") as fh:
724 fh.write(payload)
725 return dst
726
727 mp.setattr(m.storage, "fetch", fake_fetch)
728 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
729 mp.setattr(
730 m.segmenter_registry, "get", lambda name: _FakeSeg
731) # _FakeSeg persists a play_segment
732
733 r = client.post("/api/process", json={"video_path": uri, "segmenter": "x"})
734 assert r.status_code == 202 # non-local skips the os.path.exists 404 gate
735 job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
736 assert job["status"] == "done" and job["result"]["status"] == "success"
737
738 # (1) video_id is the MD5 of the FETCHED bytes (compute_video_id over the scratch
file),
739 # NOT a hash of the URI string ? catches a regression that hashes the source
ref.
740 ref_file = tmp_path / "ref_for_hash.bin"
741 ref_file.write_bytes(payload)
742 assert job["video_id"] == m.compute_video_id(
743 str(ref_file)
744) # MD5 of fetched content, not the URI
745
746 # Provenance: the videos row keeps the SOURCE URI and is marked processed.
747 rec = db.get_video(job["video_id"])
748 assert rec is not None and rec["filepath"] == uri and rec["status"] == "processed"
749
750 # (2) Queryability ? the SPA's back-half: POST /api/query {video_id} returns the
rallies.
751 segs_http = client.post("/api/query", json={"video_id": job["video_id"]}).json()[
752 "play_segments"
753]
754 assert len(segs_http) >= 1
755 assert db.query_play_segments(job["video_id"], {}) # same rows directly from the DB
756
757
758 def test_process_unknown_scheme_returns_400(env):
759 """An unsupported scheme (https/ftp/file) is a clean 400, never a 500 ? parse_uri's
ValueError

```

```

760 must not escape the request handler."""
761 client = env[0]
762 for bad in ("https://h/x.mp4", "ftp://h/x.mp4", "file:///etc/passwd"):
763 r = client.post("/api/process", json={"video_path": bad})
764 assert r.status_code == 400, f"{bad} -> {r.status_code}"
765
766
767 def test_process_local_scheme_uri_accepted(env):
768 """An explicit local://<existing file> resolves to the bare path: the existence gate
stats the
769 RESOLVED key (not the raw URI) so it is accepted, and the segmenter sees the
stripped path."""
770 client, db, tmp_path, mp = env
771 vid = _video(tmp_path, "ls.mp4")
772 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
773 mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
774 _CaptureSeg.last_path = None
775 # Forward-slash local:// URI (parse strips the scheme to the bare path).
776 uri = "local://" + str(vid).replace("\\", "/")
777 r = client.post("/api/process", json={"video_path": uri, "segmenter": "x"})
778 assert r.status_code == 202
779 assert _CaptureSeg.last_path == str(vid).replace("\\", "/")
780
781
782 def test_process_local_scheme_uri_missing_still_404(env):
783 """local://<missing> still 404s ? the existence gate runs on the resolved local
key."""
784 client = env[0]
785 r = client.post(
786 "/api/process", json={"video_path": "local:///no/such/ls-missing.mp4"}
787)
788 assert r.status_code == 404
789
790
791 def test_process_hosted_mode_rejects_nonlocal_uri(env):
792 """Hosted mode (allowed_video_root set) must reject a bucket/Drive URI ? a tenant
can't point
793 the engine at an arbitrary bucket. Pins the security contract instead of an
incidental side effect."""
794 client, db, tmp_path, mp = env
795 root = tmp_path / "tenant"
796 root.mkdir()
797 mp.setattr(m.global_config.security, "allowed_video_root", str(root))
798 for bad in ("gs://b/x.mp4", "s3://b/x.mp4", "gdrive://Sharing/x.mp4"):
799 r = client.post("/api/process", json={"video_path": bad})
800 assert r.status_code == 400, f"{bad} -> {r.status_code}"
801 # A local file UNDER the root is still accepted (passes validation; processing
mocked elsewhere).
802 under = root / "ok.mp4"
803 under.write_bytes(b"hashable")
804 mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
805 mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
806 r = client.post("/api/process", json={"video_path": str(under), "segmenter": "x"})
807 assert r.status_code == 202
808
809
810 def test_cli_nonlocal_input_fetches_and_keeps_uri(tmp_path, monkeypatch):
811 """The CLI --video-path block (a second M2 call-site) routes a bucket URI through
fetch: the
812 segmenter sees a LOCAL path and the DB keeps the source URI."""
813 db = Database(str(tmp_path / "cli.db"))
814 cfg = AppConfig()

```

These tests run with `tmp\_path` as cwd? No ? the tests run from the repo root, and `req.video\_path` carries the raw URI (`gs://bucket/videos/x.mp4`) into `emit\_indexer\_report`, which is NOT mocked in these tests. That's why `gs://bucket/videos/x.report.md` got created at the repo root. The `cli.\*` artifacts come from `test\_cli\_nonlocal\_input\_fetches\_and\_keeps\_uri`.

Now let me find the variable holding the resolved local path at both `main.py` call sites, so the fix can use it instead of the raw URI. Let me check the surrounding context for the API handler (around line 850-960) and the CLI (around 1650-1730).

## 42. user (2026-06-29T17:03:36.018Z)

```
10:import tempfile
115:# fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
309: video_path: str
468: later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
520: Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
523: cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as a
job failure
526: ref = storage.parse_uri(uri)
528: with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
529: local = storage.fetch(
659: # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to
this box.
661: input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
664: f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
683: req.video_path,
695: video_uri=req.video_path,
801: # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
802: # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
804: local_video = storage.acquire_local_input(
805: req.video_path, getattr(config, "storage", None)
807: video_id = compute_video_id(local_video)
812: logger.info(f"Running validations for {req.video_path}...")
813: val_results, val_context = registry.run_all_with_context(local_video, config)
834: req.video_path,
849: video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
890: video_id, local_video, req.player_pool, req.max_frames
953: video_path=req.video_path,
987: validate_input_path(req.video_path, allowed_root=allowed_root)
991: req.video_path, getattr(request.app.state.config, "storage", None)
998: # fetches it (a fetch miss fails the job loudly). validate_input_path above still runs
for every
1003: status_code=404, detail=f"Video file not found at '{req.video_path}'"
1015: req.video_path,
1223: # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches the
row by id so
1283: # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-scratch
for a URI (the
1312:def run_cli_diagnose_clip(video_path: str, timestamp: float):
1315: print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
1322: results = registry.run_all(video_path, cfg)
1336: cap = cv2.VideoCapture(video_path)
1518: if not args.video_path:
1537: f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1552: args.video_path, [(args.trim_start, args.trim_end)], out_name
1560: if not args.video_path:
1563: run_cli_diagnose_clip(args.video_path, args.debug_clip)
1644: if args.video_path and not args.server and not args.app:
1645: print(f"[CLI] Processing video file: {args.video_path}")
1646: # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched
```



```

to scratch.
1648: # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1651: input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1656: print(f"[FAIL] Video file not found: {args.video_path}")
1658: local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1660: video_id = compute_video_id(local_video)
1666: local_video, global_config
1673: video_id, args.video_path, status, [r.model_dump() for r in val_results]
1705: video_id, local_video, max_frames=args.max_frames
1725: video_path=args.video_path,
1741: if args.app or args.server or not args.video_path:

```

#### 43. assistant (2026-06-29T17:03:42.432Z)

Both call sites have `local\_video` available ? the resolved local filesystem path from `storage.acquire\_local\_input(...)`. This is the byte-on-disk path; the report convention is to write "next to the video," so `local\_video` is the correct, already-resolved value.

The worker (`\_\_main\_\_.py:298`) already passes `local\_video`. The fix is to make `main.py` consistent: pass `local\_video` instead of `req.video\_path`/`args.video\_path` as `video\_path` to `emit\_indexer\_report`.

However, I need to verify: (1) `local\_video` is in scope at both `finally` blocks; (2) the DB provenance still keeps the source URI (it does ? separate `add\_video`/`update` calls use `req.video\_path`); (3) whether `local\_video` could itself be a URI for a missing local file. Let me check `acquire\_local\_input`.

#### 44. user (2026-06-29T17:03:44.833Z)

```

backend/storage/__init__.py:48:def resolve_input_ref(raw: str, storage_config: Any = None) ->
StorageRef:
backend/storage/__init__.py:72:def acquire_local_input(
backend/storage/__init__.py:48:def resolve_input_ref(raw: str, storage_config: Any = None) ->
StorageRef:
backend/storage/__init__.py:72:def acquire_local_input(

```

#### 45. user (2026-06-29T17:03:47.338Z)

```

48 def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
49 """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
50 config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
51 callers at the API/CLI boundary catch that and return a clean client error (never a
500).
52
53 Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
54 ``local:///`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
55 sc = _storage_dict(storage_config)
56 uri = resolve_input_uri(
57 raw,
58 input_backend=sc.get("input_backend", "local"),
59 input_root=sc.get("input_root", ""),
60)
61 return parse_uri(uri)
62
63
64 def input_is_local(raw: str, storage_config: Any = None) -> bool:
65 """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
66 plain on-disk file the caller can stat/validate directly. False for a bucket / Drive

```

```

URI
67 that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
68 ``resolve_input_ref``)."""
69 return resolve_input_ref(raw, storage_config).backend == "local"
70
71
72 def acquire_local_input(
73 raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
74) -> str:
75 """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
76
77 Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
78 byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
79 non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
80 path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
81 regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
82 worker's ``cmd_infer`` already ships."""
83 sc = _storage_dict(storage_config)
84 ref = resolve_input_ref(raw, sc)
85 if ref.backend == "local":
86 return (
87 ref.key
88) # identity ? no temp, no copy, byte-identical to passing the path directly
89 scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
90 return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
91
92
93 def _backend_class(name: str):
94 """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
95 if name == "local":
96 from backend.storage.local import LocalStorage
97
98 return LocalStorage
99 if name == "s3":
100 from backend.storage.s3 import S3Storage
101
102 return S3Storage
103 if name == "gcs":
104 from backend.storage.gcs import GCSStorage
105
106 return GCSStorage
107 if name == "gdrive":
108 from backend.storage.gdrive import GDriveStorage
109
110 return GDriveStorage
111 if name == "gdrive-api":
112 from backend.storage.gdrive_api import GDriveApiStorage
113
114 return GDriveApiStorage
115 raise ValueError(
116 f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
117)
118
119
120 def get_storage(
121 backend: Optional[str] = None,
122 *,

```



```

837)
838 response = {
839 "video_id": video_id,
840 "status": "failed",
841 "message": "Video failed validation checks.",
842 "results": [r.model_dump() for r in val_results],
843 }
844 return response
845
846 # 2. Run segmenter & indexer
847 logger.info("[API] Video passed checks. Segmenting and indexing...")
848 db.add_video(
849 video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
850)
851
852 segmenter_cls = segmenter_registry.get(seg_name)
853 if not segmenter_cls:
854 # Submit-time validation already 400s unknown names; this is the defensive
855 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
856 available = list(segmenter_registry.list_available().keys())
857 response = {
858 "video_id": video_id,
859 "status": "failed",
860 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
861 "results": [r.model_dump() for r in val_results],
862 "play_segments": [],
863 }
864 return response
865
866 logger.info(f"[API] Using segmenter: {seg_name}")
867 notify(f"segmenting ({seg_name})")
868 segmenter_actual = seg_name
869 # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only

```

#### 48. assistant (2026-06-29T17:03:58.822Z)

`local\_video` (line 804) is assigned before the `try` (line 828), so it's always in scope at the `finally`. Same structure at the CLI site (line 1658 before the `try` at 1662).

Now let me confirm the CLI site's try/finally bounds and that `local\_video` precedes the try.

#### 49. user (2026-06-29T17:04:00.653Z)

```

1655 if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1656 print(f"[FAIL] Video file not found: {args.video_path}")
1657 return
1658 local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1659
1660 video_id = compute_video_id(local_video)
1661 was_reingest = db_instance.get_video(video_id) is not None
1662
1663 # Validate (with-context variant surfaces ffprobe_meta for the report)
1664 print("[CLI] Validating...")
1665 val_results, val_context = registry.run_all_with_context(
1666 local_video, global_config
1667)
1668 failures = [r for r in val_results if not r.passed]
1669
1670 # Save video status
1671 status = "failed" if failures else "processed"
1672 db_instance.add_video(
1673 video_id, args.video_path, status, [r.model_dump() for r in val_results]

```

```
1674)
1675
1676 seg_name = args.segmenter or getattr(
1677 getattr(global_config, "indexing", None), "default_segmenter", "motion"
1678)
1679 segmenter_actual = None
```

#### 50. assistant (2026-06-29T17:04:04.655Z)

Confirmed. `local\_video` (1658) precedes the try at 1662. Both call sites are safe.

Now let me check the current `.gitignore` and confirm there's nothing already covering these scheme dirs, and check the existing `job\_worker.py` caller (line 78) since it also passes `video\_path`.

#### 51. user (2026-06-29T17:04:05.355Z)

```
1 # Byte-compiled / optimized / DLL files
2 __pycache__/
3 *.py[cod]
4 *$py.class
5 *.pt
6
7 # Local environment secrets
8 .env
9 *.env
10 .env.*
11
12 # Provenance stamp baked into a deploy tarball at build time (deploy/gcp/README.md
section 4);
13 # never a tracked source file -- generated per-deploy, must not ride into a commit.
14 /GIT_SHA
15
16 # C extensions
17 *.so
18
19 # Distribution / packaging
20 .Python
21 build/
22 develop-eggs/
23 dist/
24 downloads/
25 eggs/
26 .eggs/
27 lib/
28 lib64/
29 parts/
30 sdist/
31 var/
32 wheels/
33 share/python-wheels/
34 *.egg-info/
35 .installed.cfg
36 *.egg
37 MANIFEST
38
39 # SQLite databases & local output files
40 *.db
41 *.db-journal
42 *.db-wal
43 output/
44 output_dir/
45 backend/static/highlights_*.mp4
46 backend/static/*_compiled.mp4
47
```

```

48 # Local generated artifacts (review reels, eval CSVs, e2e/proof outputs) -- regenerable,
never committed
49 artifacts/
50
51 # OS generated files
52 .DS_Store
53 Thumbs.db
54 desktop.ini
55
56 # Coverage data
57 .coverage
58 .coverage.*
59 htmlcov/
60 .pytest_cache/
61
62 # Local Claude Code config (machine-specific)
63 .claude/
64
65 # Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from
66 # ruff + pytest; never meant to be committed -- keep them out of `git status` too).
67 scratch/
68 .venv/
69
70 # Private leak-scan name-list (real player/venue names for LOCAL `tools.leak_scan
--names`
71 # scanning -- NEVER committed; CI runs the name-free structural guard).
72 .leakscan_private
73

```

## 52. user (2026-06-29T17:04:05.928Z)

```

55 WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
56 NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
57 releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
58 is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
59
60 The wiring is additive: the production segmenter path never raises this today (zero
61 behaviour change), so a job runs exactly as before. The hook is what a later slice
62 (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
63 """
64
65 def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
66 super().__init__(f"job parked for {delay_sec:.0f}s")
67 self.delay_sec = max(0.0, float(delay_sec))
68 self.progress = progress
69
70
71 def _job_to_request(job: Dict[str, Any]):
72 """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
73 re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
74 the original submit. The row stores exactly the ProcessRequest fields."""
75 import backend.main as _main
76
77 return _main.ProcessRequest(
78 video_path=job["video_path"],
79 player_pool=job.get("player_pool"),
80 max_frames=job.get("max_frames"),
81 segmenter=job.get("segmenter"),
82 # v8 compute-picker: the worker runs _execute_processing ?
71 _maybe_dispatch_remote, which
83 # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off

```

```

84 # the job row for analytics). NULL ? server default = pre-picker behaviour.
85 compute=job.get("compute"),
86)
87
88
89 def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
90 """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
91 terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
92
93 Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
94 the worker finished ? the pipeline's own verdict (validation failure etc.) lives in

```

### 53. assistant (2026-06-29T17:04:10.157Z)

`job\_worker.py:78` reconstructs a `ProcessRequest` and routes it back into `backend.main`'s processing function ? it does NOT call `emit\_indexer\_report` directly. So fixing the two `main.py` call sites covers the API job path too. The `.gitignore` has no coverage for the colon-scheme dirs.

Now let me confirm the existing stray dirs are regenerable junk (only report artifacts, no source) and that they're untracked by git (not committed).

### 54. user (2026-06-29T17:04:15.185Z)

```

=== file types in stray dirs ===
 16 json
 32 md

```

```

=== are any TRACKED by git? ===
(empty above = none tracked)

```

```

=== git status for these ===
?? gcs:/
?? gs:/
?? local:/

```

```

=== total file count ===
48

```

### 55. assistant (2026-06-29T17:04:23.450Z)

Confirmed: 48 files, all `.json`/`.md` report artifacts (16 json + 32 md = json+report.md+summary.md triples), zero tracked by git, all untracked (`??`). They are 100% regenerable junk from running the test suite ? safe to delete.

Now let me verify the exact `git ls-files` for the report files to confirm no obsreport-generated content is a source file, and double-check the `output/` dir is the canonical place reports should land for the fetched-bucket case. Let me also confirm what value `local\_video` takes for a `local://` URI to a real file (the `test\_process\_local\_scheme\_uri\_accepted` path) ? it should be the bare path, so reports land next to the real file, not at repo root.

### 56. user (2026-06-29T17:04:28.882Z)

```

local:// case -> report dir: '/tmp/pytest-of-avidullu/pytest-3/test_process_local_scheme_uri_0'
-> next to real file, NOT repo root: True
bucket case -> report dir: '/tmp/rally-input-abc123' (scratch tempdir, not repo root)
cwd = /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer

```

### 57. assistant (2026-06-29T17:04:33.254Z)

The fix is verified to work. With `local\_video` (the resolved path from `acquire\_local\_input`):  
- `local://` URI ? reports land next to the real file under `/tmp/...` (not repo root).

- bucket URI ? reports land in the scratch tempdir (not repo root).
- bare local path ? reports land next to the video, identical to today (zero behaviour change).

Now let me design the unit test. The best place is a focused test asserting `emit\_indexer\_report` / the API path never creates a scheme-prefixed dir at repo root. Let me check the existing test fixture/env setup and how tests assert cwd, and look for an existing reporting-path test to mirror.

#### 58. user (2026-06-29T17:04:34.328Z)

```
1 """TestClient coverage for the FastAPI seam (previously zero ? the compiler SyntaxError
2 that broke `import backend.main` shipped green because nothing imported it)."""
3
4 import os
5 import subprocess
6 import sys
7 from unittest.mock import MagicMock, patch
8
9 import pytest
10
11 pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
12 from fastapi.testclient import TestClient
13
14 import backend.main as m
15 from backend.config.models import AppConfig
16 from backend.infrastructure.database import Database
17 from backend.pipeline.validators.base import ValidationResult
18
19
20 class _InlineExecutor:
21 """submit() runs the fn synchronously ? async jobs complete before the response
22 is inspected (full lifecycle coverage lives in test_async_jobs.py)."""
23
24 def submit(self, fn, *args, **kwargs):
25 fn(*args, **kwargs)
26
27
28 @pytest.fixture
29 def env(tmp_path, monkeypatch):
30 db = Database(str(tmp_path / "t.db"))
31 cfg = AppConfig()
32 monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
33 app = m.create_app(cfg, db)
34 client = TestClient(app)
35 return client, db, tmp_path, monkeypatch
36
37
38 def _video(tmp_path, name="m.mp4"):
39 p = tmp_path / name
40 p.write_bytes(b"not-a-real-video-but-hashable")
41 return p
42
43
44 def _pass():
45 return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
46
47
48 def _fail():
49 return [
50 ValidationResult(
51 validator_name="blur", passed=False, message="blurry", details={}
52)
53]
54
55
```



```

56 class _FakeSeg:
57 def __init__(self, db, cfg):
58 self.db = db
59
60 def process_video(self, video_id, video_path, *a, **k):
61 sid = self.db.add_play_segment(video_id, 0.0, 5.0)
62 return [
63 {
64 "id": sid,
65 "start_time": 0.0,
66 "end_time": 5.0,
67 "duration": 5.0,
68 "metadata": {},
69 }
70]
71
72
73 def test_api_config_ok(env):
74 client = env[0]
75 r = client.get("/api/config")
76 assert r.status_code == 200
77 assert r.json()["sport"] == "badminton"
78
79
80 def test_api_config_redacts_secret_shaped_fields(env):

```

#### 59. assistant (2026-06-29T17:04:39.968Z)

I have everything I need. Note these tests don't mock `emit\_indexer\_report`, and obsreport is installed (so reports actually get written) ? which is precisely why running the suite created the stray dirs. The existing `test\_process\_bucket\_uri\_yields\_queryable\_video\_id` already drives the `gs:///`/`gcs:///` path; I just need to add an assertion that no scheme-prefixed dir appears at repo root.

Let me write the recommended test to a scratch file so I can validate it both fails-before / passes-after conceptually, and verify the repo-root path used. Let me check the repo root constant available to tests.

#### 60. user (2026-06-29T17:04:45.088Z)

```

cwd: /home/avidullu/projects/khelsutra-guru/badminton-highlight-indexer
has pyproject: True
gcs: exists at cwd? True
gs: exists at cwd? True
local: exists at cwd? True

```

#### 61. assistant (2026-06-29T17:04:48.200Z)

Everything is confirmed. The stray dirs are created relative to `os.getcwd()` (the repo root during test/CLI runs). Let me do a final check on the obsreport `RunRecorder.write` signature to ensure passing the resolved local path flows through as `video\_path` correctly (it pulls from `envelope.identity.video\_path`).

#### 62. user (2026-06-29T17:04:49.219Z)

```

128 self.envelope.recompute_verdict()
129 self._finalized = True
130 return self.envelope
131
132 def write(self, **io_kwargs: Any) -> WriteResult:
133 """Finalize then write all report files. Never raises (see io.write_report)."""
134 self.finalize()
135 kwargs: Dict[str, Any] = dict(
136 video_path=self.envelope.identity.video_path,

```

```
137 video_id=self.envelope.identity.video_id,
138)
139 kwargs.update(io_kwargs)
140 return write_report(self.envelope, self.spec, **kwargs)
141
```

**63. assistant (2026-06-29T17:04:52.933Z)**

Confirmed. `RunRecorder.write()` pulls `video\_path` from `envelope.identity.video\_path`, which is set from the `video\_path=` arg passed to `record\_run` ? `RunRecorder(...)` in `harvest.py:227`. So fixing the value passed to `emit\_indexer\_report` in `main.py` is the single surgical change.

I have all the information needed. Let me compile the complete fix plan.

**64. user (2026-06-29T17:05:47.838Z)**

Structured output provided successfully